CHANDIGARH UNIVERSITY

PROJECT REPORT

COMPLAINT MANAGEMENT PORTAL

Desktop Application Using Java, Swing, JDBC and MySQL

Submitted by

Alok Yadav

UID: [INSERT UID HERE]

Submitted to: [FACULTY/SUPERVISOR NAME]
Department: [DEPARTMENT NAME]
Academic Session: [ACADEMIC SESSION]

Chandigarh University | [CAMPUS / LOCATION]

CERTIFICATE

This is to certify that the project report entitled “Complaint Management Portal” is submitted by Alok Yadav (UID: [INSERT UID HERE]) as an academic project submission. The report describes the existing desktop application and its implementation based on the source code and evidence available at the time of preparation.

This page is a certificate template. It records no approval, evaluation, or institutional authorization until completed and signed by the appropriate university authority.

Particular	Details
Faculty / Supervisor	[FACULTY/SUPERVISOR NAME]
Designation	[DESIGNATION]
Department	[DEPARTMENT NAME]
Date	[DATE]

Signature: _____

Place: _____

STUDENT DECLARATION

I, Alok Yadav (UID: [INSERT UID HERE]), declare that this project report titled “Complaint Management Portal” has been prepared for academic submission. The report documents the existing application by examining its source files, SQL schema, configuration, and available execution evidence. Claims about implementation and testing are limited to those sources and observations.

Where execution was not performed, the corresponding feature or test is identified as not tested. Demonstration records shown in screenshots were created in an isolated report-only database and are not represented as production or personal records.

Student	UID	Date	Signature
Alok Yadav	[INSERT UID HERE]	[DATE]	_____

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty and academic staff of Chandigarh University for the learning environment and guidance that support project-based study. I also acknowledge the maintainers of the official Java, MySQL, Maven, FlatLaf, and jBCrypt documentation used to verify technical details in this report.

This acknowledgement does not name or imply a specific individual contribution. The report and student identification are solely those of Alok Yadav (UID: [INSERT UID HERE]).

Alok Yadav
UID: [INSERT UID HERE]

ABSTRACT

The Complaint Management Portal is a desktop application intended to replace fragmented complaint intake and follow-up with a shared, persistent workflow. Inspection of the project confirms a Java 17-targeted application using Swing, FlatLaf, JDBC, MySQL, and Maven. Its MVC-style packages separate views, a session-aware controller, services, data-access objects, and models.

Implemented source capabilities include account sign-in and registration, role-sensitive navigation, complaint submission and tracking, search and filters, owner editing of pending complaints, administrator assignment and status/priority/remarks updates, history and activity recording, user/category administration, dashboard summaries, and database-backed reports. Passwords are stored as BCrypt hashes; legacy SHA-256 hashes are upgraded after successful authentication. JDBC operations use prepared statements, and complaint changes that write related history are grouped in transactions.

The packaged database schema contains six related MySQL tables. A compatibility initializer creates or upgrades the schema and maps supported legacy complaint fields. Live verification in an isolated demonstration database covered schema-upgrade repeatability, login, complaint creation/editing, administrator assignment/status changes, filtering, history, and dashboard/report retrieval. Several user-management and validation paths remain untested and are not claimed as verified. Screenshots in this report are genuine captures from the running application; dashboard demonstration data is explicitly identified.

Keywords: complaint workflow, Java Swing, JDBC, MySQL, MVC, desktop application, role-based access.

TABLE OF CONTENTS

Section	Page
Certificate	2
Student Declaration	3
Acknowledgement	4
Abstract and Keywords	5
Chapter 1 — Introduction	7
Chapter 2 — Requirements and System Analysis	8
Use-Case Analysis	9
Chapter 3 — Architecture and Design	10
Workflow and Components	11
Chapter 4 — Database and JDBC	12–13
Chapter 5 — GUI and Implementation	14–16
Chapter 6 — Testing and Results	17–18
Chapter 7 — Conclusion and Future Scope	19
References	20

Pagination is set to the intended 20-page structure and verified against the rendered PDF.

CHAPTER 1 — INTRODUCTION

1.1 Background and motivation

When service concerns are reported through disconnected messages or paper records, it can be difficult to identify ownership, preserve a status history, and give the submitter a reliable progress view. This project provides a desktop workflow in which complaints are stored centrally and can be reviewed by the submitter and administrator.

1.2 Objectives

- Provide authenticated entry for regular users and administrators.
- Capture categorized, prioritized complaints with unique complaint codes.
- Support status tracking, administrative assignment, remarks, and history.
- Persist records in MySQL and derive dashboard/report values from queries.
- Separate Swing presentation from application services and JDBC data access.

1.3 Scope and intended users

A regular user can register/sign in, manage a profile, submit and review personal complaints, search/filter records, and edit or delete eligible pending complaints. An administrator can view all complaints and users, maintain categories, assign and update complaints, and access reports. The desktop client is a local GUI backed by a configured MySQL server; it is not a web or mobile application.

1.4 Boundaries

Email notifications, attachments, complaint escalation, and multi-department routing are not present in the inspected implementation and are treated as future scope. Actual application behavior depends on an available MySQL service and correctly configured credentials.

CHAPTER 2 — REQUIREMENTS AND SYSTEM ANALYSIS

2.1 Existing and proposed workflow

A manual or fragmented process can leave status and ownership unclear. The proposed portal stores the complaint record, assignment, administrative update, and activity history in MySQL so the authorized desktop screens can retrieve current information.

2.2 Functional requirements supported by source

ID	Requirement
F1	Register, sign in, sign out, edit profile, and change password.
F2	Create complaints with category, title, description, and priority; view personal records.
F3	Search by text and filter by status, priority, and category.
F4	Allow owners to edit pending complaints; enforce ownership in DAO queries.
F5	Allow administrators to view/update complaints, assign users, and add remarks.
F6	Record complaint updates/activities; provide complaint history.
F7	Manage users/categories and query dashboard/report summaries.

2.3 Non-functional requirements

Data integrity is supported by relational constraints and transactions for related complaint/history writes. Credential handling uses BCrypt for new hashes and parameterized JDBC queries for user-supplied values. UI database loading uses SwingWorker on relevant screens. Maintainability is supported by distinct app, view, controller, service, DAO, model, config, and utility packages. Availability is dependent on MySQL and local configuration.

2.4 Environment and feasibility

Area	Observed / practical requirement
Software	JDK 17+ target; Maven; MySQL 8-compatible server; Windows bundle available.
Libraries	MySQL Connector/J 9.2.0, FlatLaf 3.5.4, jBCrypt 0.4.
Hardware	Desktop/laptop capable of running a Java GUI and MySQL client; no performance benchmark was performed.
Feasibility	Technically feasible as a desktop client connected to MySQL; deployment requires database provisioning and credential configuration.

2.5 USE-CASE ANALYSIS

The diagram models only functions present in the source. User actions are scoped to the signed-in account; administrator navigation exposes additional management screens. Registration and sign-in are shared entry points.

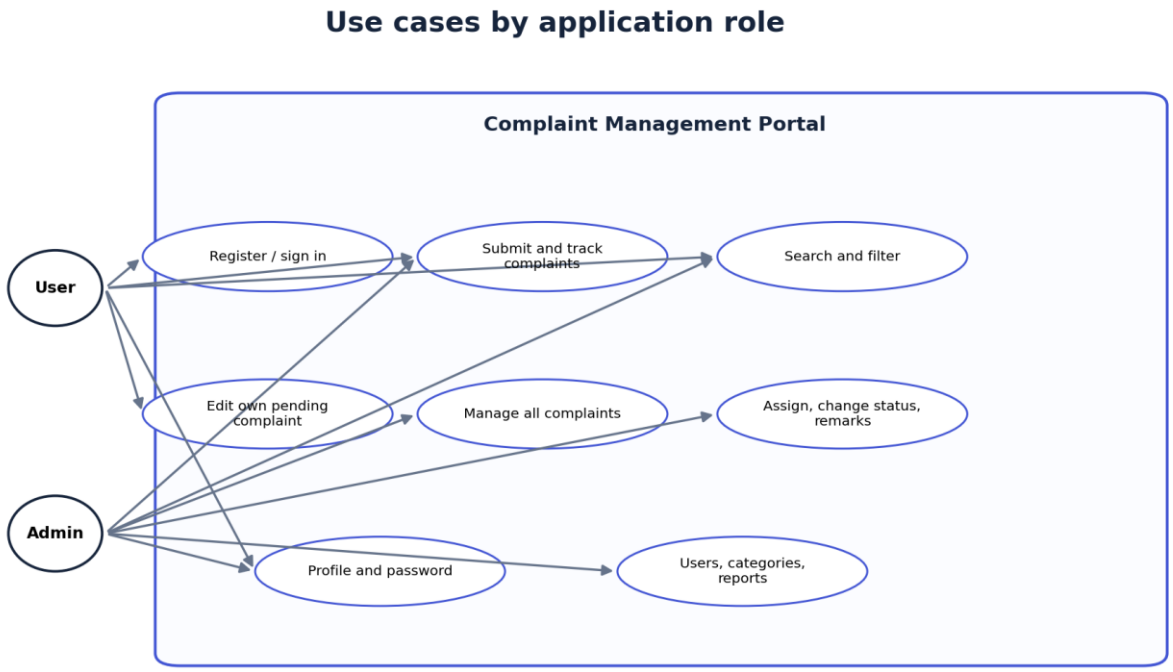


Figure 2.1. Use-case view of the existing user and administrator roles.

The authorization boundary is implemented in services and data-access queries: regular complaint retrieval includes the current user ID, whereas administrator paths operate across complaint records. The diagram does not imply external integrations or automatic notifications.

CHAPTER 3 — SYSTEM ARCHITECTURE AND DESIGN

The project follows an MVC-style separation rather than a framework-enforced MVC runtime. Swing views collect input and display results; ApplicationController exposes session-scoped operations; services apply application rules; DAOs execute JDBC statements; records/models carry data. Main composes these components and launches the login frame on the Swing event queue.

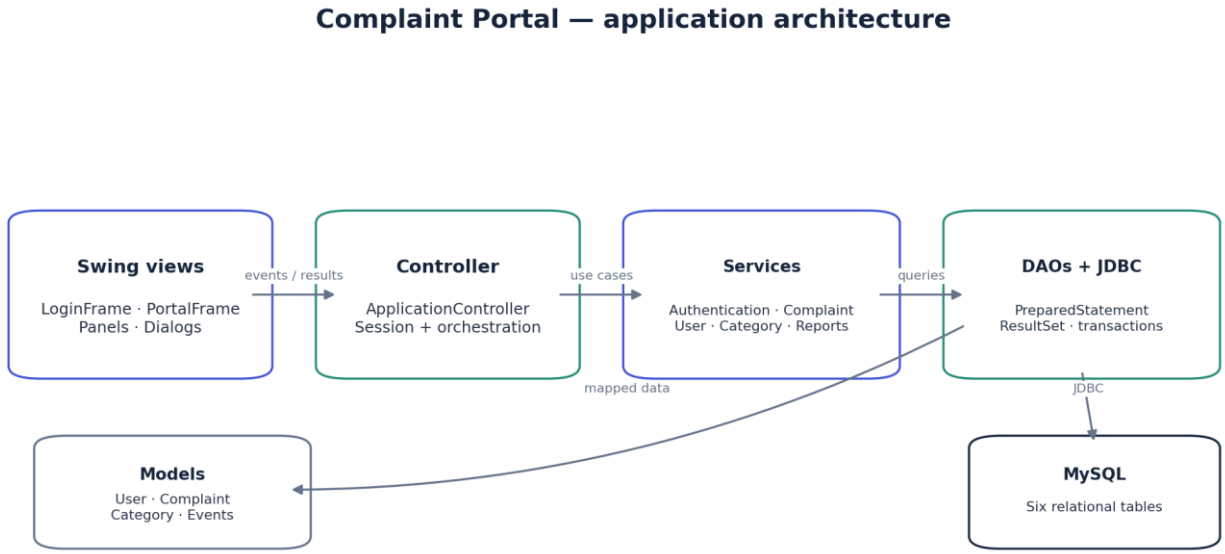


Figure 3.1. Application layers and JDBC-to-MySQL boundary.

3.1 Runtime responsibilities

Layer	Project responsibilities
View	LoginFrame, PortalFrame, panels, and dialogs; role-sensitive navigation and user feedback.
Controller	ApplicationController coordinates login, session checks, and service calls.
Service	Authentication, complaint, user, category, and report use-case logic.
DAO / config	Prepared JDBC operations, connection URL handling, schema setup/migration.
Model	User, Complaint, Category, events, dashboard and report data.

3.2 APPLICATION WORKFLOW AND COMPONENTS

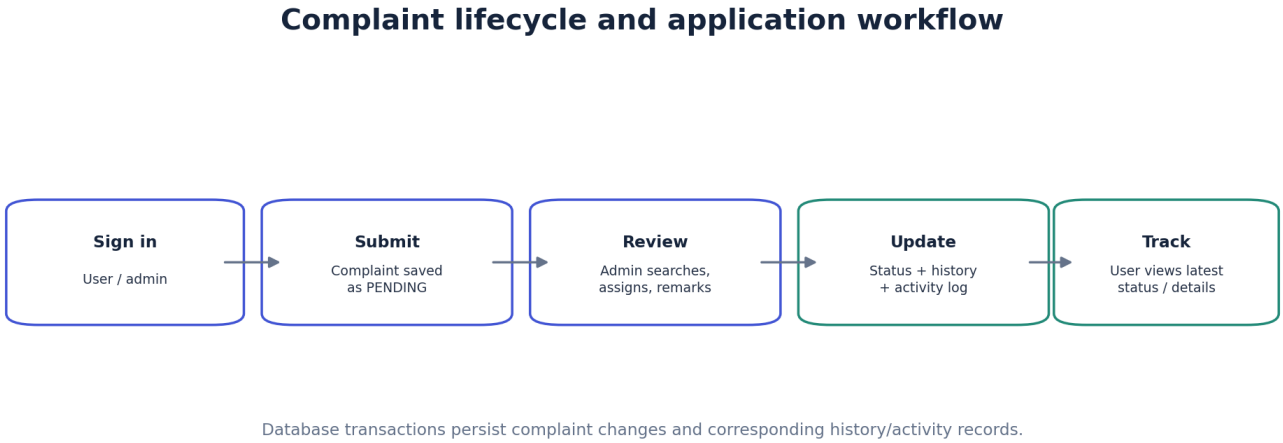


Figure 3.2. Complaint creation, administrative review, and user tracking.

Creating a complaint generates a CMP-prefixed code and commits the complaint together with an initial status update and activity log. Administrative updates persist status, priority, assignment, and remarks while recording status history and activity. A submitter's permitted edits are limited to their pending records.

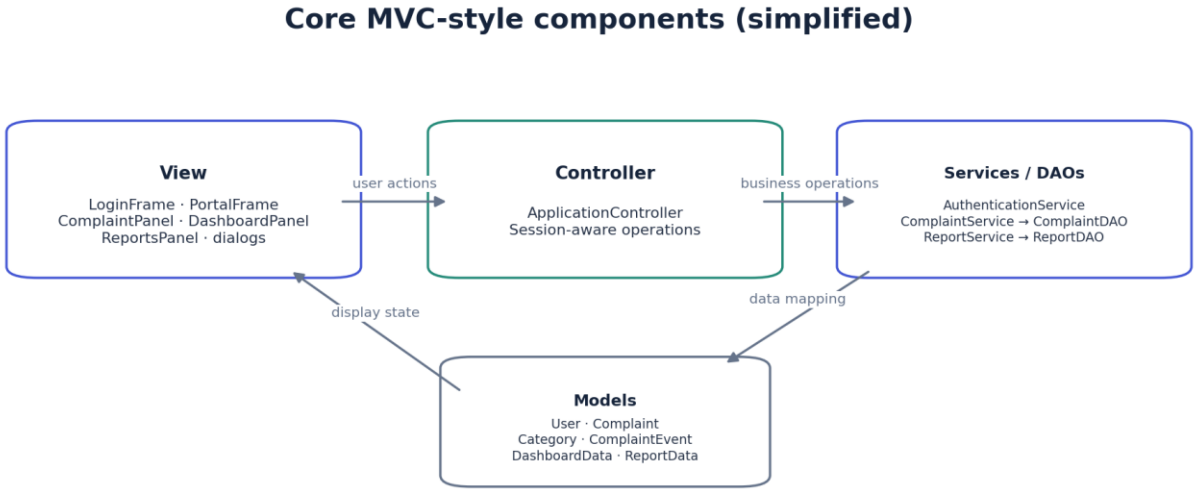


Figure 3.3. Simplified component/class view based on existing packages.

Long-running database loads are dispatched with SwingWorker on applicable screens so rendering and interaction need not wait for query completion.

CHAPTER 4 — DATABASE DESIGN AND JDBC IMPLEMENTATION

The canonical SQL resource creates the configurable database `complaint_portal` by default, with InnoDB tables, UTF-8 character data, primary/unique keys, indexes, and foreign keys. The following column inventory summarizes the shipped `database/schema.sql` definitions.

Table	Columns and types (PK / FK / notable constraints)
users	id BIGINT PK AI; full_name VARCHAR(120); email VARCHAR(190) UNIQUE; password_hash VARCHAR(60); role ENUM(ADMIN,USER); active BOOLEAN; created_at, updated_at TIMESTAMP.
complaint_categories	id BIGINT PK AI; name VARCHAR(80) UNIQUE; description VARCHAR(255) NULL; active BOOLEAN; created_at TIMESTAMP.
complaints	id BIGINT PK AI; complaint_code VARCHAR(32) UNIQUE; user_id BIGINT FK; category_id BIGINT FK; title VARCHAR(180); description TEXT; priority/status ENUM; assigned_admin_id BIGINT NULL FK; admin_remarks TEXT NULL; created_at, updated_at TIMESTAMP; resolved_at TIMESTAMP NULL.
complaint_updates	id BIGINT PK AI; complaint_id BIGINT FK; actor_id BIGINT FK; old_status/new_status VARCHAR(24) NULL; remarks TEXT; created_at TIMESTAMP.
complaint_assignments	id BIGINT PK AI; complaint_id BIGINT FK; admin_id BIGINT FK; assigned_by BIGINT FK; assigned_at TIMESTAMP.
activity_logs	id BIGINT PK AI; user_id BIGINT NULL FK; complaint_id BIGINT NULL FK; action VARCHAR(120); details VARCHAR(500) NULL; created_at TIMESTAMP.

Complaint status values are PENDING, IN_PROGRESS, RESOLVED, and REJECTED; priorities are LOW, MEDIUM, HIGH, and URGENT. Email, category name, and complaint code are unique. Timestamps use database defaults; updated_at is maintained on row updates.

4.1 ENTITY RELATIONSHIPS AND JDBC

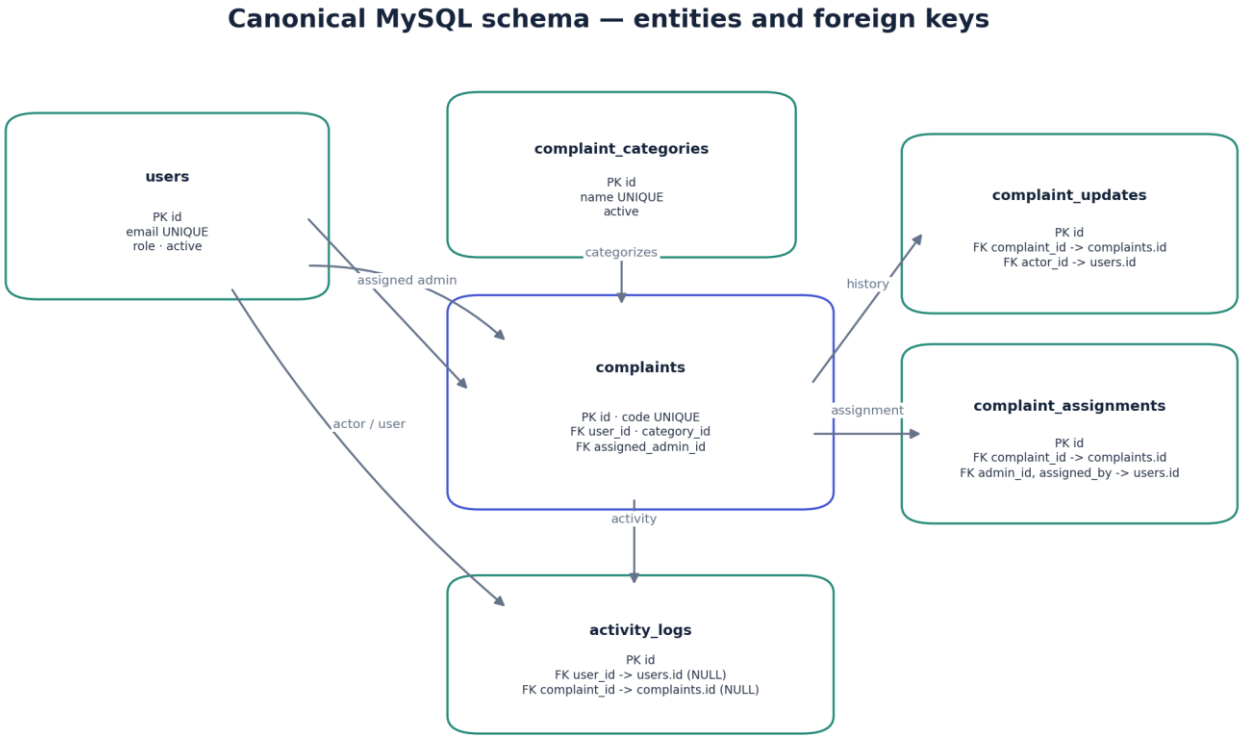


Figure 4.1. ER view derived from canonical schema foreign keys.

A user submits complaints and may be the assigned administrator; a category classifies complaints. Each complaint can have many status updates, assignment records, and activity rows. Update actors, assignment administrators/assigners, and activity users reference users. Activity user/complaint and assigned administrator links are nullable; complaint history and assignment rows cascade when a complaint is removed.

4.2 Connectivity, persistence, and compatibility

Database constructs the MySQL JDBC URL using the configured database name and opens DriverManager connections. DAOs use try-with-resources, PreparedStatement parameter binding, and ResultSet mapping. Search values are parameters rather than SQL text. Complaint create/edit/admin-update paths use transactions and roll back related writes on SQL failure.

SchemaInitializer creates the database if needed and applies the shipped schema, then upgrades compatible legacy user/category/complaint layouts: maps old subject/category and status/priority values, adds current fields, and retains legacy subject/category columns as nullable where present. The actual configured installation was a migrated legacy schema; the ER figure represents the canonical schema plus the application's supported legacy compatibility behavior.

New passwords are BCrypt hashes (cost 12). Supported SHA-256 legacy hashes are checked at sign-in and upgraded after successful authentication. Default credentials are configuration-controlled; deployment operators should replace bootstrap credentials.

CHAPTER 5 — GUI DESIGN AND APPLICATION IMPLEMENTATION

5.1 Login and registration entry

The Java Swing sign-in frame uses FlatLaf styling and provides email/password inputs, a show-password control, sign-in, and account creation. The application validates credentials through the authentication service and presents startup/database failures with an error dialog rather than switching to mock records.

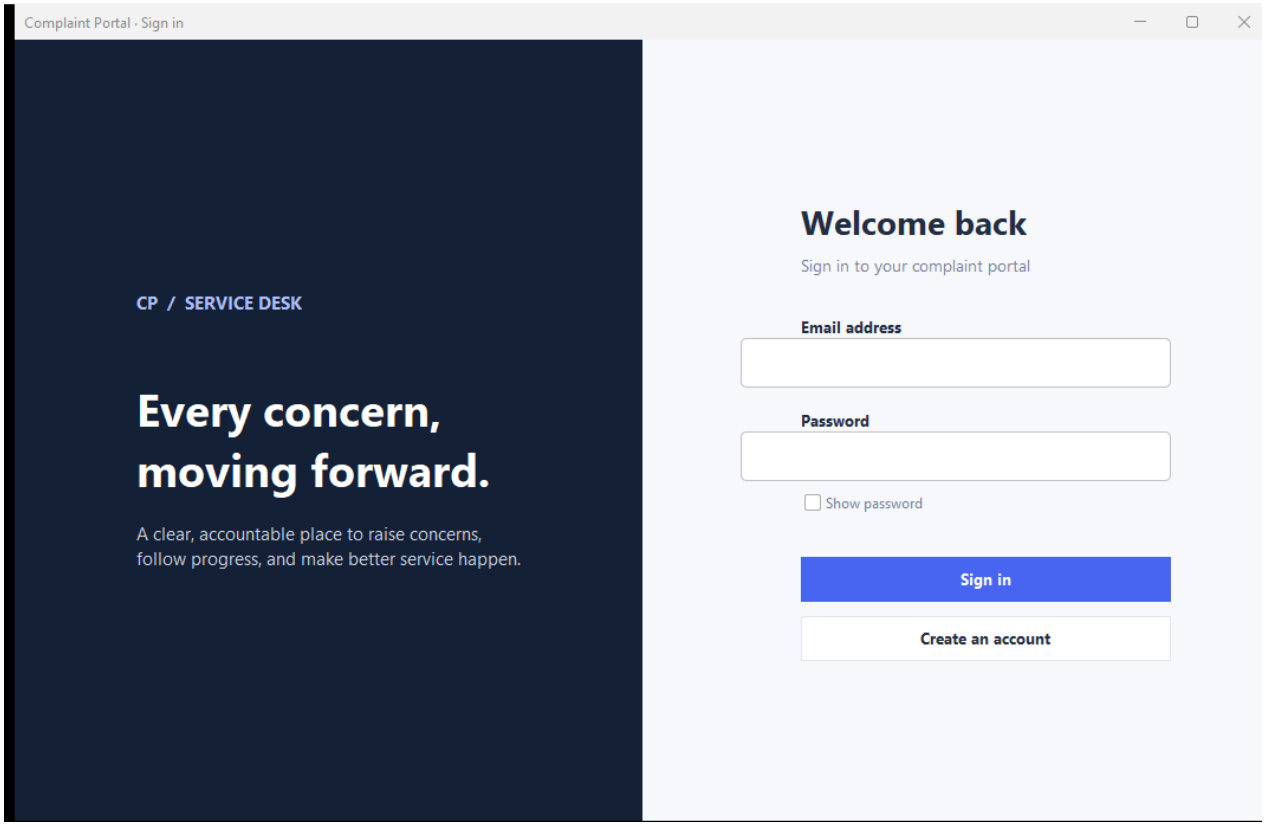


Figure 5.1. Genuine credential-free sign-in screen captured from the running desktop application.

Registration is available from the same entry screen. It is implemented in source but was not separately exercised in the report verification session.

5.2 Complaint management

The complaint-management screen presents a searchable table, category/priority/status filters, a new-complaint action, and detail/edit/delete actions for the selected record. The DAO scopes non-administrator queries to the signed-in owner. Complaint dialogs provide the available submission, detail, history, and administrator update interactions.

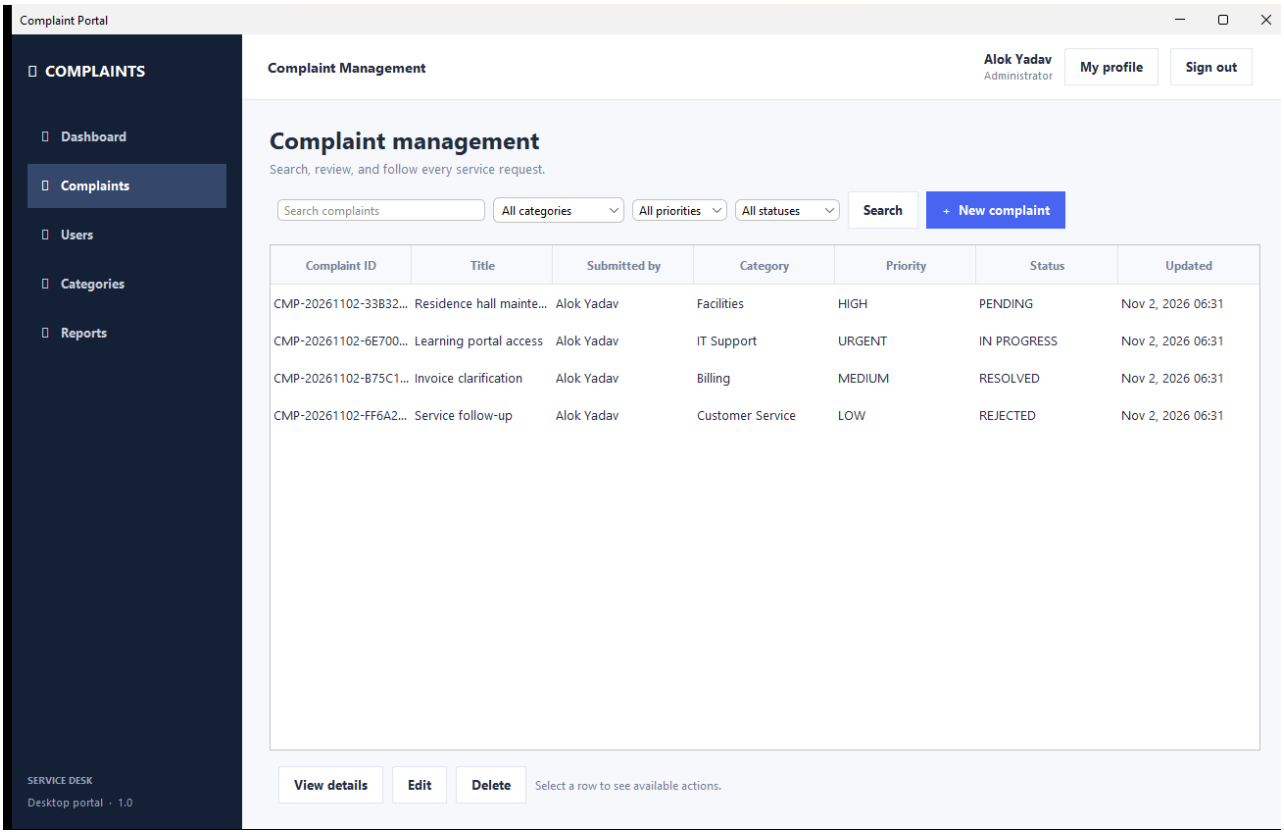


Figure 5.2. Genuine complaint list from the isolated report demonstration database.

The four visible records were generated only for screenshot and live-query verification; they are demonstration fixtures, not records from the user's configured database.

5.3 Dashboard and reporting

Dashboard and report panels obtain totals, status/category distributions, recent complaints, activity, monthly counts, and resolution measures through ReportDAO queries. User views are scoped to that user; administrator views are cross-user. CSV export is included in the report screen implementation.

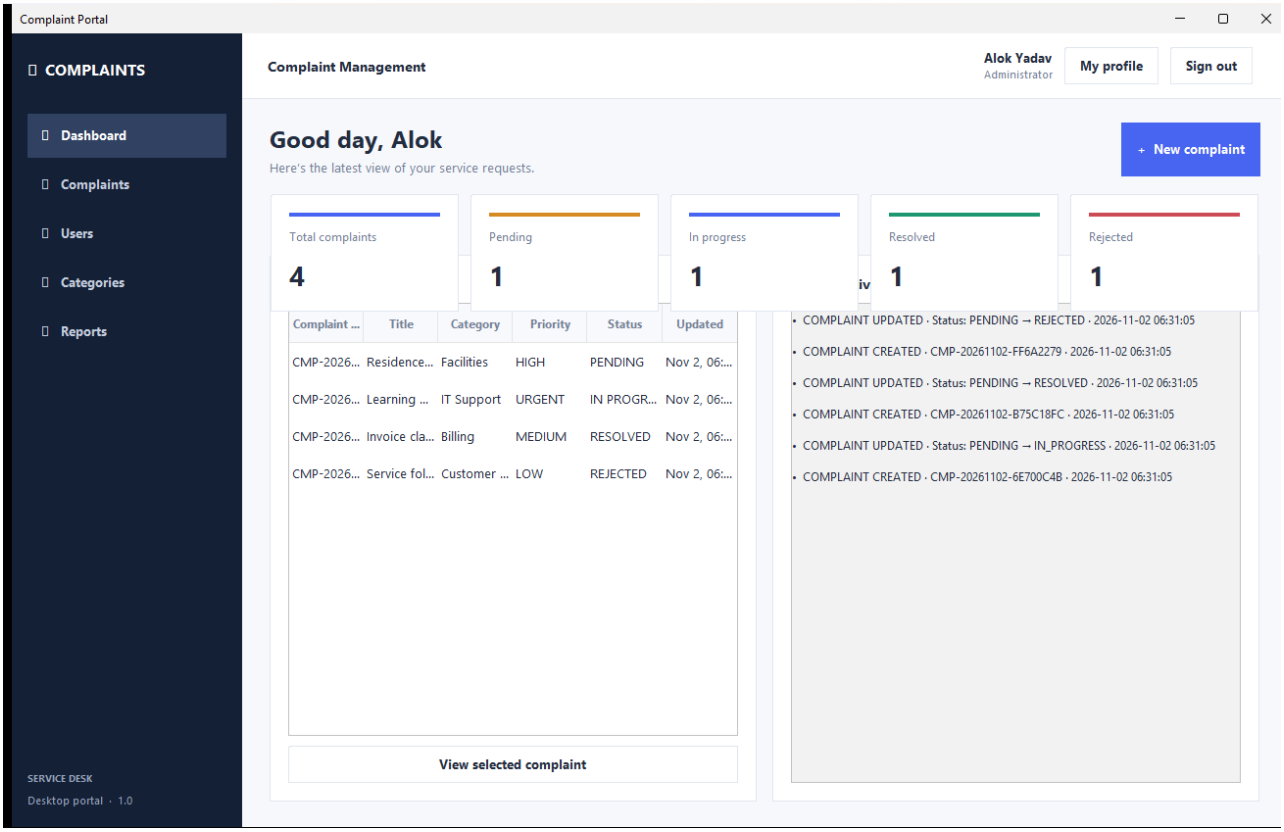


Figure 5.3. Genuine authenticated dashboard capture using four isolated demonstration complaints (one per status).

The captured dashboard shows dynamically queried counts. At the captured display size, the KPI row visually overlaps part of the lower dashboard content; this is an observed layout limitation in the running screen and is not concealed or represented as a report-rendering artifact.

CHAPTER 6 — TESTING AND RESULTS

Testing status below reflects observed execution in the report session and existing build evidence. PASS means the stated path was executed against an isolated MySQL demonstration database; NOT TESTED means no direct verification is claimed.

ID	Test scenario	Expected result	Actual result	Status
T01	Build/package application	Executable package is produced	Existing Maven package completed	PASS
T02	Initialize/migrate schema twice	Repeat run is safe	Idempotency verified on report DB	PASS
T03	Administrator sign-in	Valid credentials open portal	Authenticated demo session opened	PASS
T04	Create complaint	Persist unique code and initial history	DAO flow and database record verified	PASS
T05	Edit pending complaint	Owner changes persist	Edit flow verified	PASS
T06	Admin assignment/status update	Assignment/status/history persist	Update flow verified	PASS
T07	Search and filter	Matching records returned	Filtered query verified	PASS
T08	Complaint history/dashboard	Database values displayed	History and dashboard/report queries verified	PASS
T09	Registration/profile/password	Account/profile changes persist	Not exercised in this session	NOT TESTED
T10	Category/user administration	Admin changes persist	Not exercised in this session	NOT TESTED
T11	Invalid input/authorization edges	Validation and access limits hold	No exhaustive negative test performed	NOT TESTED

6.1 RESULTS, EVIDENCE, AND LIMITATIONS

Observed results

The isolated MySQL smoke test confirmed repeatable schema migration, authenticated access, persisted complaint creation and editing, administrator assignment/status update, filtered search, history retrieval, and database-backed dashboard/report results. Temporary verification data was kept separate from the configured application database. The source and runtime screenshots provide additional evidence of login, complaint browsing, and dashboard screens.

Evidence classification

Classification	Examples
Implemented and verified	Application packaged/launched; isolated DB sign-in; schema upgrade repeat; complaint persistence/update paths exercised as listed in T04–T08; dashboard/report DAO retrieval.
Implemented, not fully verified	Registration, profile/password change, category and user administration, complete role matrix, invalid-input behavior, and all UI controls.
Future enhancement	Email notification, attachments, escalation, multi-department workflows, and web/mobile client.

Limitations

The evidence does not establish production deployment, load capacity, measured performance, or exhaustive security testing. The dashboard screenshot exposes a visual overlap at its captured size. Before operational deployment, test all authorization edges, validation/error cases, account lifecycle flows, and UI resizing/display scaling; review bootstrap account configuration and database privileges.

CHAPTER 7 — CONCLUSION AND FUTURE SCOPE

The inspected project delivers a Java desktop complaint workflow backed by MySQL. The application connects user submission and tracking with administrator review, assignment, status updates, and persisted activity/history. The layered code structure, relational constraints, prepared JDBC statements, password hashing, and database-backed dashboard/report queries provide a concrete foundation for a small service desk.

Verification confirms selected end-to-end paths in an isolated demonstration database, but not every visible feature or negative case. The captured dashboard's card overlap and the untested registration, profile, category, and access-control edges should be addressed through additional UI validation and test coverage before wider deployment.

Future scope

- Correct responsive dashboard layout and validate common display scaling/window sizes.
- Add automated service/DAO tests and broader authorization, validation, and migration regression suites.
- Introduce notifications, attachment handling, escalation policies, and departmental routing if required.
- Extend analytics and accessibility after validating operational needs and privacy controls.

These items are proposals, not capabilities claimed in the current application.

REFERENCES

[1] Oracle. Java SE 17 API Specification — Swing package. https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/javax/swing/package-summary.html

[2] Oracle. JDBC Basics. https://docs.oracle.com/javase/tutorial/jdbc/basics/

[3] Oracle. JDBC Database Access. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/

[4] Oracle. Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/

[5] MySQL. MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/

[6] Apache Software Foundation. Maven Guides. https://maven.apache.org/guides/

[7] FormDev Software. FlatLaf Documentation. https://www.formdev.com/flatlaf/

[8] Mindrot. jBCrypt library source and documentation. https://github.com/jeremyh/jBCrypt

[9] Chandigarh University. Official university logo asset. https://www.cuchd.in/home-assets/img/logo.png

[10] Complaint Management Portal project source: pom.xml, database/schema.sql, and Java packages under src/main/java/com/complaints (audited for this report).

Technology references were consulted for verification. Project claims and schema details were checked against the local project source; sources [1]-[9] provide technology and branding references.